

A self-applied argument of knowledge for the provenance of a Lean development

what a verifier can learn about a machine-checked build
without being shown the code

Abstract

An artefact produced by a formal-methods pipeline — a theorem, a report, a rendered figure — carries no evidence of its own provenance. A reader is asked to believe that some source really was built with some toolchain, that the build really was accepted by a kernel, that the proof really used no axiom beyond a disclosed list, and that the artefact really is what that run emitted. Publishing the source settles the question and destroys any confidentiality; publishing nothing settles nothing.

This paper describes a machine-checked specification of the middle option, and then applies it to itself. The specification separates the public statement of a run — pinned toolchain, pinned dependency lock, pinned source-closure digest, pinned `.olean` closure, pinned elaborated statement type, disclosed axiom closure, disclosed artefact digest — from the private witness — the proof term, the sources, the build log, the intermediate artefacts — and defines a relation $R(E, x, y, w, c)$ that an argument of knowledge is over. Around that relation the development supplies: a simply typed calculus with a type checker and a semantic soundness theorem; a lifting of that checker into a rank-1 constraint system, with completeness (an honest derivation yields a satisfying assignment) and soundness (an explicit extractor turns any satisfying assignment back into a derivation); an arithmetization of the certificate layer into a 25-constraint circuit; and backends with proved soundness, completeness and hiding, together with the separations that show the notions are distinct.

The self-application is the new part. This repository measures itself, states the resulting public statement in Lean, discharges every conjunct of the relation for its own run, compiles that run into both circuits, and proves an end-to-end theorem about itself: `self_end_to_end`. Its most self-referential clause is that any satisfying assignment of the circuit lifted out of the proof checker implies this project's own specification — the meaning assignment under which the circuit is read sends the pinned atoms to propositions this development proves, so the trusted base of the circuit's soundness theorem is the development itself. Everything is checked by Lean `leanprover/lean4:v4.28.0`: 84 modules, 23,391 lines, 1044 theorems, 0 occurrences of `sorry` and 0 `axiom` declarations, with the axiom closure of the headline theorem measured as `Classical.choice`, `Quot.sound`, `propext`. The same circuit is then run through the argument stack it was built for: §9 compiles it to flat gates, verifies it recursively by the universal verifier, commits to the private assignment and proves possession of it, and shows the published artefact compatible with every private message. What remains unproved is declared, not hidden: 51 trust gaps, each annotated on the declaration that carries it and each listed in §10.

Contents

1	The problem	2
2	Public and private	2
3	The specification and the relation	3

4	The type layer	4
5	Lifting the checker into a circuit	4
6	The certificate layer as a constraint system	4
7	Arguments of knowledge over the circuit	5
8	The self-application	5
8.1	Measurement	5
8.2	The statement, in Lean	6
8.3	The two circuits	7
8.4	End to end	7
9	The zkPoP stack over the self run’s proof circuit	7
10	What is not claimed	9
11	Reproducing	12

1 The problem

Consider a pipeline that builds a Lean development, checks it, measures it, and renders a figure. The figure is published. What does its reader know?

Nothing, in the strict sense. The figure is a sequence of bytes. Every claim about how it came to exist — “lake build exited zero”, “the kernel accepted the proof”, “the only axioms used were `propext`, `Classical.choice` and `Quot.sound`”, “the scorecard hashes to this value” — is a claim about a process the reader did not witness. The usual remedy is to publish the source and the build instructions and invite the reader to re-run them. That works, and it forfeits every reason one might have had not to publish: an unreleased library, a proof one intends to submit elsewhere, a client’s code base.

The alternative this development formalises is an *argument of knowledge*: a public statement about a run, together with a proof that somebody knows a witness satisfying a relation, where the witness — the Lean proof term, the sources, the build artefacts — is never disclosed. Three things have to be got right for such a claim to mean anything.

- (i) **The environment must be public.** “Lean accepted this proof” is meaningless relative to an unpinned environment: an environment can contain a declaration asserting anything. The toolchain, the dependency lock, the source closure, the compiled closure, the elaborated statement type and the trusted axiom list are therefore all part of the public statement.
- (ii) **The relation must be falsifiable.** A relation that every witness satisfies proves nothing. Each layer below is accompanied by a theorem exhibiting an instance it *rejects*.
- (iii) **The gap between the model and reality must be stated.** A formalisation of a cryptographic pipeline is not a cryptographic pipeline. Everything this one idealises is annotated, in the code, on the declaration that idealises it, and collected in §10.

2 Public and private

The split is the design. Write E for the pinned environment, x for the run configuration, y for the published artefact, w for the witness and c for a commitment to the witness.

public	private
E : toolchain digest, dependency-lock digest, source-closure digest, <code>.olean</code> closure digest, elaborated statement type $H(T)$, backend pin, trusted axioms, trusted extensions	the Lean proof term the source closure itself the build log the intermediate artefacts
x : source digest, commit, phase seed, scorecard digest, disclosed axiom closure	the four certificates the commitment opening
y : the artefact (here, its content identifier)	
c : a commitment to w	

Two structural facts make the split load-bearing. The run configuration is *bound* to the environment: `RunConfig.BoundTo` requires $x.projectSourceDigest = E.projectSourceDigest$, and a verifying build certificate forces it, so the code that was built is the code the environment commits to. And the published commitment is tied to the prover’s actual witness by a conjunct of the relation, so a bundle cannot be replayed with a different witness.

3 The specification and the relation

A *deployment* D is a choice of four interfaces: a hash scheme, a ground truth (a deterministic renderer plus the three facts a run claims), an attestation verifier and a proof checker. Everything above is stated once, parametrically, and reused by every instantiation.

Definition 3.1 (the specification, `Deployment.Spec`). $D.Spec\ x\ y$ is the conjunction of: the build passed; the kernel closure matches the disclosed axiom list; the measured scorecard matches; and y is the deterministic render of x .

Each of the first three is presented twice: as a *certificate* (a conjunction of equalities on public and private data, mechanically checkable) and as a *fact* about the world, with an explicit soundness step (`CertificateSoundness`) between them. The one irreducibly external check is the builder attestation.

Definition 3.2 (the relation, `Provenance.Relation`). $R(E, x, y, w, c)$ holds when

- (a) the witness’s encoded proof checks in E , against the pinned declaration type, as a proof of $D.Spec\ x\ y$;
- (b) x is bound to E ;
- (c) the build, axiom, fitness and render certificates all verify;
- (d) the published commitment opens to this witness;
- (e) $y = D.render\ x$.

Conjunct (a) is the interesting one: it is not “the prover asserts the specification” but “the prover holds an object that a pinned checker accepts, in a pinned environment, as a proof of the specification”. Unconditional consequences are proved directly from R : the artefact is pinned (`Relation.unique_output`), the built source is the pinned source (`Relation.projectSource_eq`), the build exited zero, the attestation verifies. The relation is proved satisfiable and proved falsifiable (`Relation.not_vacuous`), which rules out both ways such a specification can be empty.

A verification bundle is a public statement plus a backend proof. Its consequences are theorems about *any* accepted bundle: acceptance yields the specification (`VerificationBundle.spec`), and possession of a kernel-checked proof of it (`VerificationBundle.knowsLeanProof`); a bundle disclosing an untrusted axiom, an unbound config or the wrong artefact is rejected.

4 The type layer

“The proof checks” has to mean something checkable. The development supplies a concrete calculus rather than an oracle: a de Bruijn term language with constants for previously checked declarations, function types and pairs; a typing relation; a type-inference algorithm with soundness and completeness; a reduction engine with type preservation; and a semantic soundness theorem.

Theorem 4.1 (soundness of the calculus, `Typed.sound`). *If every constant of the pinned declaration environment denotes a true proposition under a meaning assignment m — written $\text{SigTrue } S \ m$ — then every closed term of type T witnesses $\text{interp}_m(T)$.*

On top of it, `kernelChecker` is an actual checking function, and the interface obligations `CheckerSoundness` and `CheckerPinned` become theorems about that function rather than hypotheses.

The honest limit is stated where it lives: the calculus is a simply typed fragment, with no universes, no dependent types and no definitional unfolding. It is a model of “a kernel accepted this”, not Lean’s kernel.

5 Lifting the checker into a circuit

One does not lift a circuit out of a proof; one lifts it out of the *checker*, and the proof becomes the witness. Concretely: a derivation is flattened into a trace of rows, each row is packed into one cell over a finite alphabet, and the per-row admissibility checks — generated from public data alone — become a one-hot constraint system.

Write P for the public parameters (pinned declarations, statement, row bound, packing radix). The generator emits `proofCircuit( $P$ )`, a system over any field, and the two directions are proved:

Theorem 5.1 (completeness, `Lift.circuit_complete`). *A derivation of the pinned statement, of at most the public row bound, yields a satisfying assignment.*

Theorem 5.2 (extraction, `Lift.circuit_extract`). *From any satisfying assignment an explicit extractor reads back a term that type-checks in the pinned environment against the pinned statement.*

Two corollaries matter for what a verifier learns. First, falsifiability: if the statement has no derivation, the circuit has no satisfying assignment (`circuit_unsat_of_unprovable`). Second, indistinguishability of proofs: two derivations of the same statement satisfy literally the same circuit (`circuit_indistinguishable`), so the public data does not depend on which proof was used. What the lift does *not* do is hide the proof: the satisfying assignment encodes it. Hiding is the argument system’s job, §7.

Binders are handled by bracket abstraction: combinators K and S are realised as terms of the calculus, an abstraction algorithm is proved type-preserving, and the resulting term lies in the application-only fragment the lift accepts.

6 The certificate layer as a constraint system

The certificate layer arithmetizes directly. A `FieldEmbedding` sends digests injectively into a field; the public instance and the four certificates occupy 22 wires; and the verifier’s equality core becomes a predicate in a small arithmetizable language, compiled by an equality gadget (indicator bit, inverse hint, booleanity) into rank-1 constraints.

Theorem 6.1 (the circuit says what the verifier says, `r1cs_iff_core`). *Under the range hypotheses a deployment discharges with range checks, an assignment agreeing with the layout satisfies the compiled system with output wire 1 exactly when the arithmetized core of the certificate verifiers holds.*

Nothing is lost by the split into circuit and residual (`verifies_iff_core_and_residual`), and Phase X reduces the residual further, until the only conjunct not decided by constraints is the builder attestation — which is external by nature.

The same predicate is also compiled into a Clean-style circuit monad with witness generators, region markers and a formal-circuit discipline in which each gadget carries its own soundness and completeness proofs. Measured on the provenance predicate:

compilation	advice wires	constraints	top degree
general	24	25	12
optimised	0	12	1

Both rows are `decide`-checked in the development, not estimated. The optimisation compiles an equality into a single degree-one constraint and allocates no advice at all.

7 Arguments of knowledge over the circuit

A backend is an abstract verifier for public statements. The development proves properties of backends rather than assuming them: `BackendSound` (acceptance implies the relation), `BackendComplete` (an honest prover is accepted), `PerfectlyHiding` (the published proof object is a function of the public statement alone), and their probabilistic counterparts — `PerfectZK` and `KnowledgeSoundWith  $\varepsilon$` over an explicit finite probability layer, computational indistinguishability against a Boolean-circuit distinguisher of bounded size, and asymptotic versions with a security parameter, negligible error and fuel-metered extraction.

The notions are shown to be distinct rather than merely defined: the transparent backend is sound and complete but provably not hiding, and its zero-knowledge error is at least $1/2$ against every simulator; the ideal backend is sound, complete and perfectly hiding at once; and a prover is exhibited that is perfectly indistinguishable against the gate-free distinguisher class while every simulator is statistically far from it, so computational hiding does not collapse to the statistical notion.

For the non-interactive presentation there is a Σ -protocol layer with completeness, special soundness with the extractor given explicitly, the Fiat–Shamir transform and the reprogramming step that extracts from two accepting transcripts with distinct challenges; and a deployed statement in which the relation is “satisfies the compiled constraint system, opens the commitment, and renders to these bytes”.

8 The self-application

Everything above is generic. This section instantiates it on the repository that contains it.

8.1 Measurement

`tools/self_manifest.py` walks the repository and records, with SHA-256: the toolchain pin, the dependency lock, the canonical listing of the project’s own source closure (99 files), the canonical listing of the compiled `.olean` closure (77 files), and the measured scorecard `introspection/fitness.json`. It writes the result as a Lean module, `RequestProject/Provenance/Self/Manifest.lean`, and the scorecard as JSON.

`tools/ExportStatement.lean` adds two measurements taken from the compiled environment itself: the pretty-printed elaborated type of the self statement, whose digest is the $H(T)$ the environment pins, and the result of `Lean.collectAxioms` on the headline theorem, which becomes the disclosed axiom closure.

The run’s repository identifier is a *content* identifier — `dcaa381be601...`, the domain-separated digest of the four closure digests — and deliberately not a version-control commit id. A commit id names the commit that contains the measurement, so no artefact measured here could print one and still be reproducible; a digest of the contents is fixed by the contents alone. That is a small instance of the general obstruction the whole self-application has to work around, and §11 says how.

measurement	value
toolchain digest	0xdb7bb24b756d...cd3f3028
dependency-lock digest	0x84965c68cec2...dcd1bb31
source-closure digest	0x02f357c1fd2a...790232d0
.olean closure digest	0xd47cf3488e76...49068b44
scorecard digest	0xe0cb75c89d13...a02c2430
elaborated statement type $H(T)$	0xc946030a5fb1...982545bf
build-log digest	0x88832a20f6d4...d8f386a3
paper source digest	0xa418b7f80559...17677743
repository content id	0xdcaa381be601...23a91093
phase seed	20250825
disclosed axiom closure	<code>Classical.choice, Quot.sound, propext</code>

Three exclusions keep the measurement well founded. The manifest module is excluded from the closure it certifies — a file cannot contain its own digest. The scorecard is excluded, because the manifest records its digest. The built PDF is excluded, because the manifest records *its* digest, and the paper does not print it; so the loop *sources* $\rightarrow$ *manifest* $\rightarrow$ *paper* $\rightarrow$ *PDF digest* $\rightarrow$ *manifest* reaches a fixed point in one extra pass.

8.2 The statement, in Lean

`Self/Statement.lean` builds the environment `selfEnv` and the run config `selfConfig` out of those numbers, the four certificates of the run, and a deployment `selfDeployment` whose hash is the sponge of Phase I, whose attestation verifier is the signature/enclave gate of Phase IV, and whose checker is the de Bruijn kernel checker of Phase II. The run is then discharged conjunct by conjunct:

theorem	what it establishes
<code>self_buildOk</code>	the build certificate verifies, attestation included
<code>self_axiomsOk</code>	every disclosed axiom is on the trusted list
<code>self_fitnessOk</code>	the scorecard certificate verifies
<code>self_renderOk</code>	the artefact digest matches the config
<code>self_spec</code>	this repository satisfies <i>D.Spec</i>
<code>selfSigTrue</code>	the checker’s trusted base is true <i>of this run</i>
<code>self_relation</code>	$R(E, x, y, w, c)$ holds of the honest witness
<code>selfTrustAssumptions</code>	every idealised hypothesis is discharged
<code>self_bundle_exists</code>	the honest bundle verifies
<code>self_bundle_spec, self_bundle_knows</code>	acceptance buys the specification, and a proof of it
<code>self_perfectly_hiding</code>	the proof object is a function of the public statement

`selfSigTrue` is where the self-reference first bites: the checker’s pinned declaration environment has one constant, whose meaning is the specification of this run, and its truth is discharged by `self_spec` — a theorem of the same development.

8.3 The two circuits

`Self/Circuit.lean` fills the 22 wires of §6 with the measured digests (Figure 1) and proves, at that instance: the compiled system is satisfiable with output wire 1 (`self_r1cs_satisfiable`); a satisfying assignment plus the named residual reconstitutes the relation (`self_relation_of_r1cs`); and replacing the source-closure digest in the build certificate by a foreign one makes the system unsatisfiable — *no* assignment, not merely no honest one (`self_r1cs_unsat_of_foreign_source`). The same three facts are proved for the Clean-style circuit, and for the deployed non-interactive statement, for every random oracle.

`Self/Types.lean` does the corresponding thing one level up. The pinned declaration environment of the self run has 5 constants — the four certificate lemmas and the assembly lemma that turns them into the specification — the statement is their conclusion, and the honest proof term is the fourfold application, 9 rows at radix 9 (the type table has exactly 9 rows, so this is the tightest radix the parameters admit). The alphabet then has 177,150 letters and the lifted system has at most 2,541,955,111,209 constraints: one-hot lifting is quadratic in the alphabet, which is exactly why the 25-constraint certificate circuit, and not this one, is what a deployment would run.

Theorem 8.1 (the self-referential step, `self_lift_sound_interp`). *Let m be the meaning assignment sending the 5 pinned atoms to the propositions of this repository’s run. Then `SigTrue` holds of m — by `self_buildOk`, `self_axiomsOk`, `self_fitnessOk`, `self_renderOk` and `self_spec` — and therefore every satisfying assignment of the lifted circuit implies $D.\text{Spec } x \ y$: this project’s own specification.*

So the verifier checks constraints, and what it thereby learns is a theorem about the project that generated them, while the assignment that satisfies them — the proof — stays with the prover.

8.4 End to end

Theorem 8.2 (`Provenance.Self.self_end_to_end`). *For this repository’s measured environment, run `config` and artefact digest: every trust assumption of the deployment is discharged; the run satisfies the specification; the honest bundle verifies; acceptance of any bundle implies the specification and possession of a kernel-checked proof of it; the published proof object is a function of the public statement alone and is perfectly simulatable; the certificate circuit is satisfied at the measured instance and unsatisfiable at the tampered one; and the lifted proof circuit is satisfied by the honest derivation and, from any satisfying assignment, implies the specification.*

The theorem’s own axiom closure is measured, not assumed: running `Lean.collectAxioms` on it returns `Classical.choice`, `Quot.sound`, `propext`, and the build-time audit `#gap_audit` additionally fails the build unless the annotated trust gaps reachable from it are exactly the sixteen it declares.

9 The zkPoP stack over the self run’s proof circuit

§5 ends with an honest limit: the satisfying assignment of the lifted circuit *is* the derivation, so the lift turns the proof into a witness — it does not hide it. Hiding, and a verifier that can itself be run as a circuit, are the business of an argument system on top.

The self instance: this repository on the wires

22 wires of Provenance.Arith.layout, filled with the measured digests of this repository

visualization/data/self-instance.json

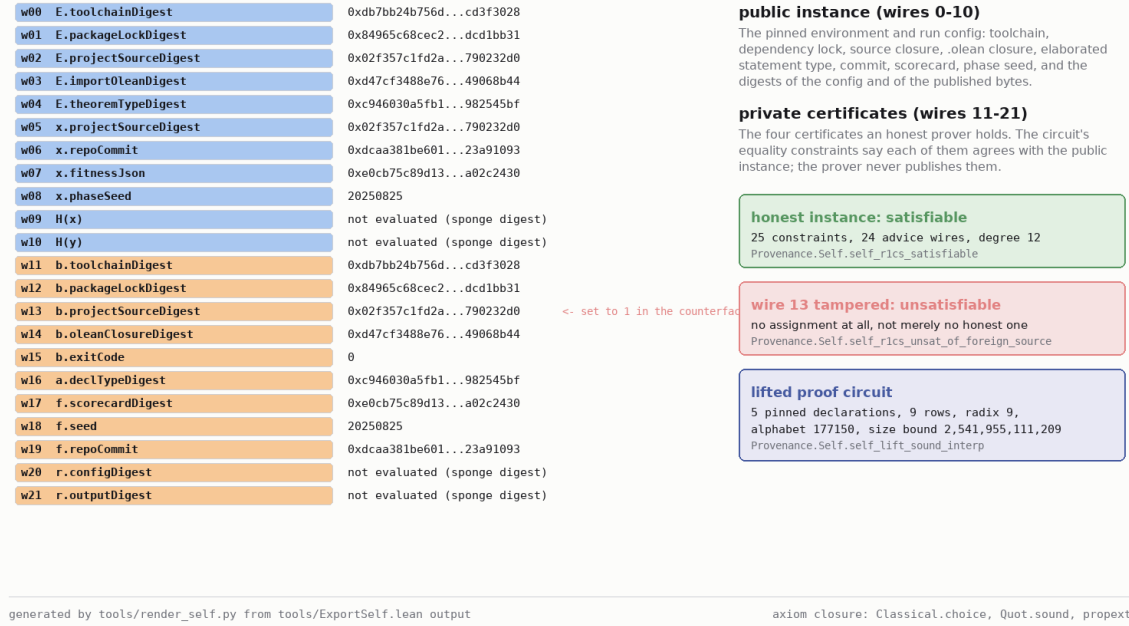


Figure 1: The 22 wires of the measured instance. Wires 0–10 are the public statement, wires 11–21 the private certificates. Two wires are sponge digests the development never evaluates. Setting wire 13 to a foreign value makes the system unsatisfiable. Drawn from visualization/data/self-instance.json, which the compiled Lean development writes.

The RequestProject/ZkPoP/ library supplies one: a polynomial-system language with a Tseitin flattener (flatSys_sat_iff), the flat verifier expressed again as a constraint system (univ_two_level), a Pedersen commitment with proved perfect hiding and a binding reduction, and a three-move protocol with completeness and special soundness. This section is that stack applied to the circuit of §8 — to this development's own proof.

The bridge. A rank-1 constraint $A \cdot B = C$ becomes the single polynomial $A \cdot B - C$, which vanishes exactly when the constraint holds, and a system becomes the list of those polynomials (ZkPoP.sysPoly). The translation is exact: an assignment satisfies the polynomial system exactly when it satisfies the rank-1 system (ZkPoP.sysPoly_sat_iff), so nothing is accepted or rejected in the move that was not accepted or rejected before. A wire bound is read off the system's own variable list, which is what the flattener needs in order to allocate scratch wires (ZkPoP.sysPoly_Below), and the flattened circuit's own bound — what the universal verifier needs — is ZkPoP.flatSys_WiresLt. All four are theorems about an arbitrary system over an arbitrary field; the self run's system, with its 2,541,955,111,209 constraint bound, is an instance and is never evaluated.

The instance. Writing cs for the self run's lifted system, Lifted.circuit cs is its flat circuit and Lifted.next cs the flattener's final wire counter. Then:

Theorem 9.1 (ZkPoP.SelfApply.self_zkpop_end_to_end). *For any generators $g = t \cdot h$ of the commitment scheme and any prover randomness:*

- (i) *the honest derivation of this repository's specification, extended to the scratch wires, admits an outer attestation — an assignment witnessing that the universal verifier accepted it;*

The self-proving loop

the repository measures itself, states the claim, compiles it to constraints, and publishes an argument

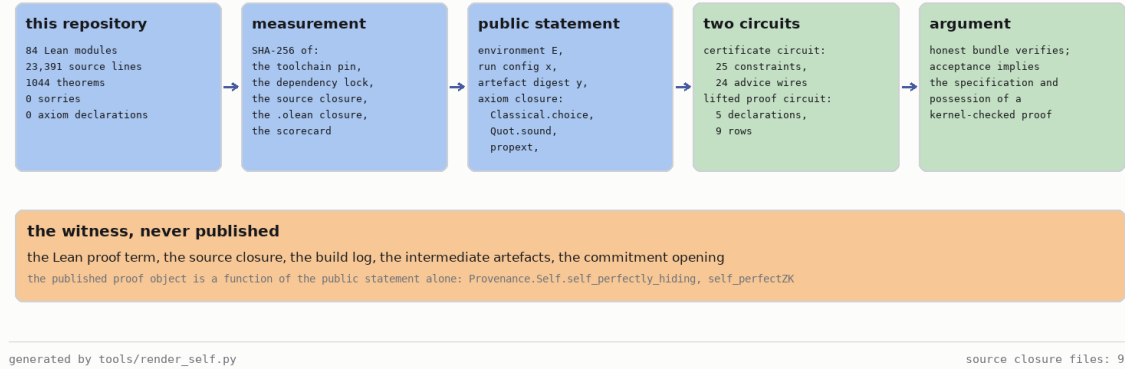


Figure 2: The self-proving loop: the repository measures itself, states the claim, compiles it to constraints from public data alone, and publishes an argument. The witness — proof term, sources, build log, intermediate artefacts, commitment opening — is never published.

- (ii) any outer attestation of any inner assignment implies both that this project’s run satisfies the provenance specification and that the extractor reads back a proof term typing against the pinned statement;
- (iii) the published transcript verifies;
- (iv) a second accepting challenge on the same announcement extracts an opening of the commitment — the prover really holds the private assignment;
- (v) the published artefact is compatible with every private message: for any alternative message there are opening and prover randomness giving a literally identical artefact.

Clause (ii) is §8’s self-referential step carried through one level of recursion: the verifier now checks gates emitted by a verifier, and what acceptance still buys is a theorem about the project that generated them. Clauses (iv) and (v) are what §5 left to the argument system: possession is extractable, and the artefact does not say which derivation was used — which, with `self_lift_indistinguishable` (different derivations of the pinned statement satisfy the same circuit), closes the gap between “a proof exists” and “this proof was used”.

What this is not. Clause (v) is the deterministic-artefact form of hiding — every alternative private message explains the identical published object — and not a distributional zero-knowledge statement; the notions are kept apart deliberately, as in §7. The committed field element is a public Horner packing of the assignment, and no injectivity is claimed for it: the commitment binds the packed value. The calculus underneath is still the simply typed fragment, and the field is still the rationals, so no proof size or prover cost is claimed. The build-time audit records that `self_zkpop_end_to_end` reaches exactly nine declared gaps — the eight of the lifted self circuit, together with `lift.witness-not-hiding` — and that the bridge theorems reach none.

10 What is not claimed

The development is free of `sorry` and adds no `axiom`, and that is a weaker statement than it looks: everything a formalisation of this kind cannot prove has to go somewhere. Here it

goes into declared gaps, each attached by an attribute to the declaration that carries it, each reachable from a headline theorem only if that theorem’s audit lists it. Two are specific to the self-application: the digests are entered measurements, and the artefact is represented by its content identifier rather than by its bytes.

id	kind	what is missing
<code>abstract.attestation</code>	external	attestation validity is opaque in the abstract deployment
<code>abstract.axiom-closure</code>	external	whether <code>Lean.collectAxioms</code> on the target returns exactly the published closure is an opaque predicate here
<code>abstract.build-passed</code>	external	whether the build exited 0 is an opaque predicate: an observation about a process, not a proposition Lean can decide
<code>abstract.digest-encoding</code>	primitive	the structural encoding of an abstract digest is opaque, so its canonicity is only ever a hypothesis
<code>abstract.digest-type</code>	primitive	the digest type of the abstract deployment is opaque: nothing is known about it
<code>abstract.fitness</code>	external	whether the measured scorecard hashes to the published value is an opaque predicate here
<code>abstract.hash</code>	primitive	the hash of the abstract deployment is opaque
<code>abstract.lean-checks</code>	external	the Lean proof checker of the abstract deployment is opaque: this is the ZK-friendly type-checking problem, deliberately not reinvented
<code>abstract.renderer</code>	external	the deterministic renderer of the abstract deployment is opaque: the bytes a real pipeline emits are not computed in Lean
<code>arith.field-embedding</code>	assumption	digests enter the field through an injective embedding with a declared numeric bound; the range conditions on numeric wires are hypotheses standing in for range-check constraints
<code>arith.residual</code>	scope	the arithmetization covers the equality core of the certificate verifiers; the builder attestation is left outside the circuit and is carried as an explicit hypothesis
<code>attestation.external</code>	external	validity of a builder attestation is an interface field: whether the signing service really ran the build is decided outside this development
<code>axioms.fixed-width</code>	scope	the axiom list conditions are decided in circuit only at public, fixed block widths: the number of axiom and extension wires is public, and equal length is a hypothesis
<code>axioms.name-embedding</code>	assumption	declaration names enter the field through an embedding whose injectivity is a theorem only for the exhibited embedding into the rationals
<code>certificates.sound</code>	assumption	that a verifying certificate implies the fact it certifies is a hypothesis, carried by every theorem that turns certificates into ground truth
<code>checker.interface</code>	external	the ZK-friendly Lean type checker is an interface, not an implementation; this file fixes the shape of the obligation only
<code>checker.pinned</code>	assumption	that the checker only accepts proofs of the declaration pinned by the environment digest is a hypothesis
<code>checker.sound</code>	assumption	soundness of the pinned checker – kernel soundness, faithfulness of the proof encoding, and soundness of the pinned axiom base – is a hypothesis
<code>clean.degree-syntactic</code>	model	degree is a syntactic measure of the emitted polynomial, not a cost model for any proving system; no claim is made about prover or verifier cost or proof size
<code>demo.rational-field</code>	scope	the worked instances are over the rationals, which is not a deployment field, and no claim is made about prover or verifier cost or proof size
<code>deployed.transparent</code>	leak	the deployed argument of knowledge is transparent: its response is the satisfying assignment, as <code>r1csSigma_reveals_witness</code> states; nothing here is zero knowledge

id	kind	what is missing
<code>encoding.opaque-bytes</code>	scope	a proof encoding is an uninterpreted byte string: nothing in this layer ties its bytes to an actual Lean proof term
<code>extractor.fuel</code>	model	extraction cost is fuel metered: the budget is the procedure's own step counter, not the running time of a fixed universal machine
<code>fs.random-oracle</code>	model	the oracle is an arbitrary function, so what is proved is the logical content of forking – reprogramming at the committed point extracts a witness – and not a probability bound in the random oracle model
<code>ground.external</code>	external	the three facts a run claims – the build exited 0, the kernel closure matched, the scorecard hashed – are supplied by the deployment; Lean never observes the build
<code>hash.ideal</code>	assumption	collision freeness of the hash, and canonicity of the digest encoding, are idealised as injectivity and assumed as hypotheses; no primitive is proved to satisfy them
<code>kernel.fragment</code>	scope	the checked calculus is a simply typed fragment – binders, pairs and pinned constants – with no universes, no dependent types and no definitional unfolding; it is not Lean's kernel
<code>lattice.order.B</code>	model	the order of the Baby Monster is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.order.Co1</code>	model	the order of Co1 is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.order.Fi23</code>	model	the order of Fi23 is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.order.J1</code>	model	the order of J1 is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.order.M11</code>	model	the order of M11 is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.order.M12</code>	model	the order of M12 is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.order.M22</code>	model	the order of M22 is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.order.M23</code>	model	the order of M23 is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.order.M24</code>	model	the order of M24 is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.order.Monster</code>	model	the order of the Monster is a constant of the classification, entered as a numeral; only its prime factorisation is checked here
<code>lattice.psl-order-formula</code>	model	the order of $\text{PSL}(n,q)$ is entered as the standard formula, not derived from a Lean construction of the group; it is cross-checked against the exceptional isomorphisms $\text{PSL}(2,4) = \text{PSL}(2,5) = A_5$, $\text{PSL}(2,9) = A_6$ and $\text{PSL}(3,4) = \text{PSL}(4,2) = A_8$, whose orders are proved here
<code>lift.combinator-count</code>	leak	the public parameters built by the translation disclose how many combinator instances a proof used, which is a leak about the proof's shape
<code>lift.implicational</code>	scope	the lift covers the implicational fragment: with the combinator declarations a derivation using implication introduction lifts, but pairing, definitional unfolding and universes stay outside
<code>lift.witness-not-hiding</code>	leak	the satisfying assignment still encodes the proof verbatim: this layer makes the proof a witness, it does not hide it
<code>minichecker.calculus</code>	model	MiniChecker is a Hilbert-style implicational calculus with a placeholder byte serialisation; its soundness theorem is about that calculus and not about Lean
<code>reference.model</code>	model	the reference deployment is a model, not a candidate implementation: its hash is transparent, its attestation verifier accepts everything, and its ground truth is defined by the existence of verifying certificates

id	kind	what is missing
<code>self.artifact-by-digest</code>	model	the published artefact is represented by its 32-byte content identifier rather than by its bytes: the renderer returns the measured SHA-256 of <code>paper/self-zk-provenance.pdf</code> , so <code>renderMatch</code> binds the run to that digest and not to a re-execution of the rendering pipeline
<code>self.entered-measurements</code>	external	the digests of this repository – toolchain, dependency lock, source closure, olean closure, statement type, scorecard, artefact – are measured outside Lean by <code>tools/self_manifest.py</code> and entered as numerals; Lean records them, it does not compute SHA-256 of the files
<code>shhs.textual-catalog</code>	external	the catalog is produced by a textual parse of the Lean sources (<code>tools/system_catalog.py</code>), not by the elaborator: kinds, dependencies and invariants such as <code>no-sorry</code> are entered measurements, and Lean checks what follows from them, not that they describe the files correctly
<code>shhs.view-opening</code>	scope	a view's checks are local to its payload: nothing here proves that the payload really is the projection of the catalog with the published source digest, so <code>verifyView_implies_global</code> carries that as an explicit hypothesis
<code>signature.unforgeable</code>	assumption	unforgeability of the signature scheme is a hypothesis of every theorem that rejects a forged attestation
<code>sponge.collision-free</code>	assumption	collision freedom of the compression function is an assumption; what is proved is the reduction from a digest collision to a compression collision
<code>sponge.non-compressing</code>	model	the exhibited instance is collision free only because its compression function is injective – it does not compress, so it is not a deployment hash
<code>zk.concrete-size</code>	model	the computational model is non-uniform and concrete size: hiding is relative to a fixed gate bound, and no circuit lower bound is proved, so the exhibited separation is at the gate-free class

In particular: the exhibited hash is collision-free only because its compression function is injective, so it does not compress and is not a deployment hash; the signature scheme whose unforgeability is used is the ideal functionality; the checked calculus is a simply typed fragment and not Lean's kernel; the worked fields are the rationals, so no proof size or prover cost is claimed; the builder attestation is external by nature; and the self-application's digests are produced by a script, with `tools/self_manifest.py --check` as the re-measurement that detects drift. A green build is evidence about the artefacts, not a proof about the world.

11 Reproducing

From the project root, `bash tools/reproduce.sh` runs the whole pipeline to its fixpoint and ends by re-measuring the repository; it prints `manifest current` exactly when every recorded digest still matches what the repository now contains. The individual steps are:

step	command
build and audit	<code>lake build</code>
re-measure the repository	<code>python3 tools/self_manifest.py --check</code>
re-export the statement and axioms	<code>lean tools/ExportStatement.lean</code>
re-export the circuit data	<code>lean tools/ExportSelf.lean</code>
redraw the figures	<code>python3 tools/render_self.py</code>
rebuild this paper	<code>python3 tools/build_paper.py</code>

The order is not arbitrary. The manifest records the digest of the elaborated statement type, of the exported instance and of this PDF, while this PDF prints digests the manifest records; the pipeline therefore has to be driven to a fixpoint. It has one because nothing the paper prints depends on the PDF, because the recorded `.olean` closure omits the modules that transitively import the manifest — their compiled form changes whenever the manifest does, exactly as the manifest source cannot appear in the source closure it records — and because the repository is identified by a content digest rather than by a commit id. Each of the three is the same obstruction: nothing an artefact contains may depend on that artefact’s own name.

The build prints, for each headline theorem, the exact list of declared gaps its constant closure reaches, and fails if that list changes.

Appendix: the measured digests in full

The table in §8 abbreviates each digest. The full SHA-256 values recorded in `Self/Manifest.lean` are below; a reader who has the repository can recompute every one of them with `python3 tools/self_manifest.py --check`.

measurement	SHA-256
toolchain pin	0xdb7bb24b756d745bbde83fe92718b51bd3625dae3701ba0f598d0eedcd3f3028
dependency lock	0x84965c68cec2c5a17f6b17216a7018b1ff71ead889c9dcf0883668f5dcd1bb31
source closure	0x02f357c1fd2a39cfac0e86d7cd501edab29e09f708cb057da58242c0790232d0
<code>.olean</code> closure	0xd47cf3488e768c7969e5e451d7b66a653d55848bac7977ab7cd90a8849068b44
scorecard	0xe0cb75c89d13a0ca8066c7f3104d0ba4be894685686eff46484b7749a02c2430
elaborated statement type	0xc946030a5fb165781e678555388cbccd3e977a431b584b970cb962c0982545bf
build log	0x88832a20f6d49d7f37f2f21a609b3e6a1bce101c617bc131d1a5d4b3d8f386a3
paper source	0xa418b7f80559886c9823f6595d5eedb15283dea86b2c3f9e79ded6f217677743
repository content id	0xdcaa381be601f2e9516b5b4ca4d21913a01a3c3cf5a8997dc48cb1d923a91093

The digest of this PDF is recorded in the manifest as `paperPdfDigest` but is deliberately not printed here: a document cannot contain its own digest.

Index of the self-application

file	contents
<code>Self/Manifest.lean</code>	the measured digests (generated)
<code>Self/Statement.lean</code>	environment, config, certificates, relation, bundle, hiding
<code>Self/Circuit.lean</code>	the certificate circuit at the measured instance
<code>Self/Types.lean</code>	typing, the lifted proof circuit, the self-referential step
<code>Self/EndToEnd.lean</code>	<code>self_end_to_end</code> and its asymptotic form
<code>ZkPoP/Bridge.lean</code>	rank-1 systems into the zkPoP stack, and their wire bounds
<code>ZkPoP/SelfApply.lean</code>	the stack over the self run’s proof circuit (§9)